

Programación 1

GRAFOS

Trabajo Práctico

Alumno

Gonzalo Tapia

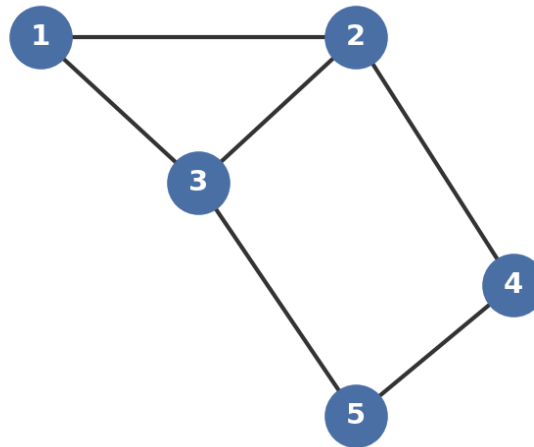
Tecnicatura Superior en Programación

Sección 1 · Conceptos teóricos

Ejercicio 1

Grafo no dirigido. Nodos: {1, 2, 3, 4, 5}. Aristas: (1,2), (1,3), (2,3), (2,4), (3,5), (4,5).

a) Dibujo.



b) Grado de cada nodo.

- Nodo 1: grado 2
- Nodo 2: grado 3
- Nodo 3: grado 3
- Nodo 4: grado 2
- Nodo 5: grado 2

Verificación: la suma de los grados es $2+3+3+2+2 = 12$, el doble de las 6 aristas.

c) Lista de adyacencia.

```
1: [2, 3]
2: [1, 3, 4]
3: [1, 2, 5]
4: [2, 5]
5: [3, 4]
```

d) Matriz de adyacencia.

	1	2	3	4	5
1	0	1	1	0	0
2	1	0	1	1	0
3	1	1	0	0	1
4	0	1	0	0	1
5	0	0	1	1	0

Es simétrica, como corresponde a un grafo no dirigido.

e) Ciclos.

- $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$
- $2 \rightarrow 4 \rightarrow 5 \rightarrow 3 \rightarrow 2$
- $1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 3 \rightarrow 1$

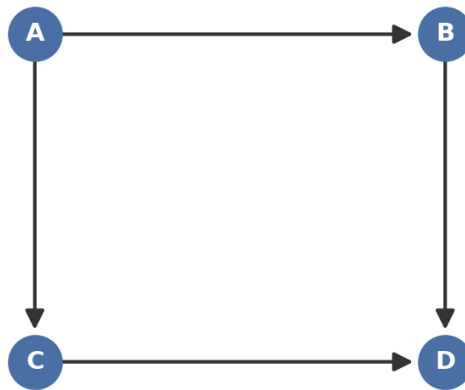
f) ¿Es conexo? Justificar.

Sí, es conexo, porque desde cualquier nodo se puede llegar a cualquier otro. Por ejemplo, para ir del nodo 1 al nodo 5 existe el camino $1 \rightarrow 3 \rightarrow 5$. Partiendo del nodo 1 se alcanzan todos los demás nodos ($1 \rightarrow 2$, $1 \rightarrow 3$, $3 \rightarrow 5$, $2 \rightarrow 4$), por lo que están todos en una sola pieza y ninguno queda aislado.

Ejercicio 2

Grafo dirigido. Nodos: {A, B, C, D}. Aristas: $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow D$, $C \rightarrow D$.

a) Dibujo (con flechas).



b) Grado de entrada y de salida de cada nodo.

- Nodo A: entrada 0, salida 2
- Nodo B: entrada 1, salida 1
- Nodo C: entrada 1, salida 1
- Nodo D: entrada 2, salida 0

c) Lista de adyacencia.

```

A: [B, C]
B: [D]
C: [D]
D: []
  
```

Al ser dirigido, cada arista se anota en una sola punta (el nodo del que sale la flecha). D queda con lista vacía porque no apunta a nadie.

d) Matriz de adyacencia (no simétrica).

	A	B	C	D
A	0	1	1	0
B	0	0	0	1
C	0	0	0	1

D	0	0	0	0
---	---	---	---	---

No es simétrica: $[A][B] = 1$ pero $[B][A] = 0$. Esto confirma que el grafo es dirigido.

e) ¿Es un DAG? Justificar.

Sí, es un DAG porque cumple las dos condiciones: es dirigido (las aristas tienen dirección, van en un solo sentido) y es acíclico (no existe ningún ciclo). El flujo va siempre hacia adelante, de A hacia B y C, y de estos hacia D; como D no tiene flechas de salida, no hay manera de volver al punto de partida siguiendo las flechas.

f) Orden topológico válido.

Hay más de un orden válido, porque B y C no dependen entre sí:

- A, B, C, D
- A, C, B, D

Ejercicio 3

Clasificación de cada situación según dirección, ciclos y conectividad.

a) Red de rutas entre ciudades de Mendoza.

- **No dirigido:** por las rutas se puede ir y volver por el mismo camino, la relación es mutua.
- **Cíclico:** tres ciudades conectadas entre sí forman un triángulo por el que se puede volver al origen.
- **Conexo:** se puede llegar de cualquier ciudad a cualquier otra pegando saltos (salvo que hubiera un pueblo totalmente aislado).

b) Cadena de mando en un cuartel militar.

- **Dirigido:** el mando va en un solo sentido (de arriba hacia abajo); un soldado no ordena a un general.
- **Acíclico:** las órdenes bajan por la jerarquía y no vuelven a subir formando un bucle. Es un DAG.
- **Conexo:** toda la estructura cuelga de un mando único, desde el cual se alcanza a cualquier subordinado.

c) Prerequisitos de materias en una carrera universitaria.

- **Dirigido:** las correlativas van en un sentido (una habilita a la otra, no al revés).
- **Acíclico:** un bucle de correlativas sería imposible de cursar. Es un DAG.
- **Desconexo:** puede haber materias sueltas (un taller u optativa) que no son correlativas de nada ni tienen correlativas, quedando como una isla separada del resto.

d) Conexiones de Wi-Fi entre dispositivos en una oficina.

- **No dirigido:** la comunicación es bidireccional, los datos van y vienen entre los dos dispositivos.
- **Cíclico o acíclico (según la topología):** si es una estrella pura (todos colgados solo del router) es acíclico; si además los dispositivos se conectan entre sí, como es habitual en una oficina, se forman bucles y resulta cíclico.

- **Conexo:** si todos están en la misma red, hay camino entre cualquier par (aunque sea pasando por el router).

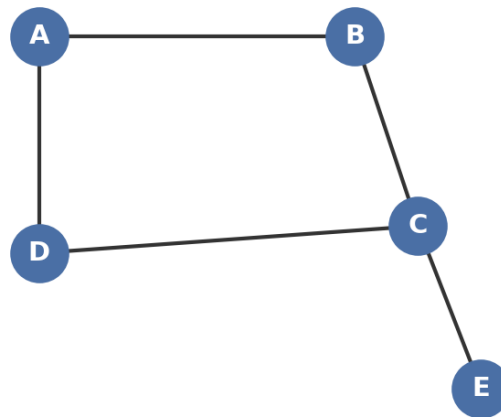
Ejercicio 4

Se parte de la siguiente matriz de adyacencia (orden de filas y columnas: A, B, C, D, E):

	A	B	C	D	E
A	0	1	0	1	0
B	1	0	1	0	0
C	0	1	0	1	1
D	1	0	1	0	0
E	0	0	1	0	0

a) Dibujo del grafo.

Aristas leídas de la matriz: A-B, A-D, B-C, C-D, C-E.



b) ¿Dirigido o no dirigido? ¿Cómo se dieron cuenta?

No dirigido, porque la matriz es simétrica: por ejemplo $[A][B] = 1$ y $[B][A] = 1$, $[C][E] = 1$ y $[E][C] = 1$. Cada casillero coincide con su espejo respecto de la diagonal.

c) Lista de adyacencia equivalente.

```

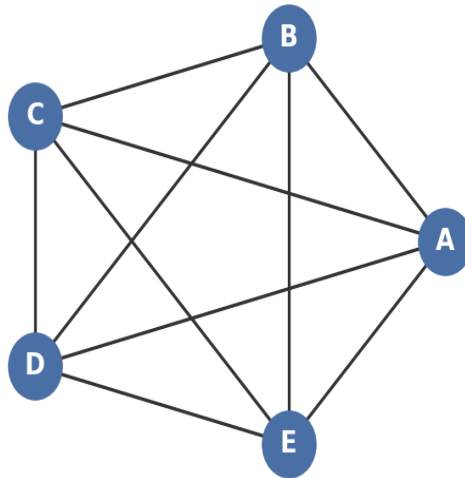
A: [B, D]
B: [A, C]
C: [B, D, E]
D: [A, C]
E: [C]
  
```

d) ¿Es conexo? Justificar.

Sí, es conexo. Partiendo del nodo más aislado (E, que tiene un solo vecino) se alcanzan todos los demás: $E \rightarrow C \rightarrow D \rightarrow A$ recorre E, C, D y A, y $E \rightarrow C \rightarrow B$ agrega B. Como desde E se llega a los 5 nodos, están todos conectados en una sola pieza.

Ejercicio 5

Grupo de 5 amigos donde cada amistad es mutua y todos son amigos entre sí (grafo completo).



a) ¿Cuántas aristas tiene el grafo?

10 aristas. Cada uno de los 5 nodos tiene grado 4, y la suma de grados es $5 \times 4 = 20$, el doble de las aristas; por lo tanto $20 \div 2 = 10$. También se obtiene con la fórmula del grafo completo $n(n-1)/2 = 5 \times 4 / 2 = 10$.

b) ¿Cuál es el grado de cada nodo?

Grado 4: cada persona es amiga de las otras 4 (todos menos ella misma).

c) Matriz de adyacencia.

	A	B	C	D	E
A	0	1	1	1	1
B	1	0	1	1	1
C	1	1	0	1	1
D	1	1	1	0	1
E	1	1	1	1	0

Todos los casilleros valen 1 excepto la diagonal, que vale 0 (un nodo no es adyacente consigo mismo).

Sección 2 · Implementación en Python

Ejercicio 6

Implementar `crear_listas_adyacencia(nodos, aristas)` y probarla con el grafo del Ejercicio 1.

```
def crear_listas_adyacencia(nodos, aristas):
    listas_adyacencia = {}
    for nodo in nodos:
        listas_adyacencia[nodo] = []
    for u, v in aristas:
        listas_adyacencia[u].append(v)
```

```

        listas_adyacencia[v].append(u)
    return listas_adyacencia

nodos = [1, 2, 3, 4, 5]
aristas = [(1, 2), (1, 3), (2, 3), (2, 4), (3, 5), (4, 5)]
resultado = crear_listas_adyacencia(nodos, aristas)
print(resultado)

```

Salida:

```
{1: [2, 3], 2: [1, 3, 4], 3: [1, 2, 5], 4: [2, 5], 5: [3, 4]}
```

Ejercicio 7

Implementar **crear_matriz_adyacencia(nodos, aristas)** y probarla con el grafo del Ejercicio 1.

```

def crear_matriz_adyacencia(nodos, aristas):
    n = len(nodos)
    matriz = [[0] * n for _ in range(n)]
    for u, v in aristas:
        i = nodos.index(u)
        j = nodos.index(v)
        matriz[i][j] = 1
        matriz[j][i] = 1
    return matriz

nodos = [1, 2, 3, 4, 5]
aristas = [(1, 2), (1, 3), (2, 3), (2, 4), (3, 5), (4, 5)]
resultado = crear_matriz_adyacencia(nodos, aristas)
print(resultado)

```

Salida:

```

[[0, 1, 1, 0, 0],
 [1, 0, 1, 1, 0],
 [1, 1, 0, 0, 1],
 [0, 1, 0, 0, 1],
 [0, 0, 1, 1, 0]]

```

*Nota: se usa `[[0]*n for _ in range(n)]` y no `[[0]*n]*n`, porque esta última crea *n* referencias a la misma fila y modificar un casillero afectaría a todas.*

Ejercicio 8

Función **grado_nodo(listas_adyacencia, nodo)** que devuelve el grado de un nodo. Probada con todos los nodos del Ejercicio 1.

```

def grado_nodo(listas_adyacencia, nodo):
    return len(listas_adyacencia[nodo])

listas = crear_listas_adyacencia(nodos, aristas)
for nodo in nodos:
    print(f"el nodo {nodo}, tiene grado {grado_nodo(listas, nodo)}")

```

Salida:

```
el nodo 1, tiene grado 2
el nodo 2, tiene grado 3
el nodo 3, tiene grado 3
el nodo 4, tiene grado 2
el nodo 5, tiene grado 2
```

El grado de un nodo es la cantidad de vecinos que tiene en su lista de adyacencia.

Ejercicio 9

Función **son_adyacentes(listas_adyacencia, nodo1, nodo2)** que devuelve True o False. Probada con los pares (1,3), (1,4) y (4,5).

```
def son_adyacentes(listas_adyacencia, nodo1, nodo2):
    return nodo2 in listas_adyacencia[nodo1]

pares = [(1, 3), (1, 4), (4, 5)]
for a, b in pares:
    print(f"¿{a} y {b} son adyacentes? {son_adyacentes(listas, a, b)}")
```

Salida:

```
¿1 y 3 son adyacentes? True
¿1 y 4 son adyacentes? False
¿4 y 5 son adyacentes? True
```

Dos nodos son adyacentes si uno figura en la lista de vecinos del otro (existe una arista directa).

Ejercicio 10

Modificar la función para grafos **dirigidos** (agregar la conexión en un solo sentido). Probada con el grafo del Ejercicio 2.

```
def crear_listas_adyacencia_dirigido(nodos, aristas):
    listas_adyacencia = {}
    for nodo in nodos:
        listas_adyacencia[nodo] = []
    for u, v in aristas:
        listas_adyacencia[u].append(v)  # solo un sentido
    return listas_adyacencia

nodos = ['A', 'B', 'C', 'D']
aristas = [('A', 'B'), ('A', 'C'), ('B', 'D'), ('C', 'D')]
resultado = crear_listas_adyacencia_dirigido(nodos, aristas)
print(resultado)
```

Salida:

```
{'A': ['B', 'C'], 'B': ['D'], 'C': ['D'], 'D': []}
```

La única diferencia con la versión no dirigida es que se elimina el segundo append: en un dirigido la lista de cada nodo contiene solo los nodos a los que apunta.

Ejercicio 11

Función **grado_entrada_salida(listas_adyacencia_dirigido, nodo)** que devuelve la tupla (grado_entrada, grado_salida). Probada con el Ejercicio 2.

```
def grado_entrada_salida(listas_adyacencia_dirigido, nodo):
    grado_salida = len(listas_adyacencia_dirigido[nodo])
    grado_entrada = 0
    for lista_vecinos in listas_adyacencia_dirigido.values():
        if nodo in lista_vecinos:
            grado_entrada += 1
    return (grado_entrada, grado_salida)

grafo_dirigido = crear_listas_adyacencia_dirigido(nodos, aristas)
for nodo in nodos:
    print(f"Nodo {nodo}: {grado_entrada_salida(grafo_dirigido, nodo)}")
```

Salida:

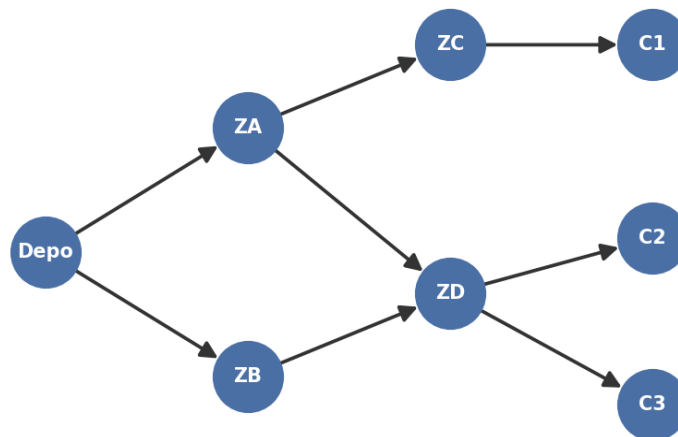
```
Nodo A: (0, 2)
Nodo B: (1, 1)
Nodo C: (1, 1)
Nodo D: (2, 0)
```

El grado de salida es el largo de la lista del nodo. El grado de entrada hay que buscarlo recorriendo todas las listas y contando en cuántas aparece el nodo.

Ejercicio 12 · Integrador

Empresa de delivery con rutas de entrega de ida solamente.

a) Representación como grafo dirigido.



Nodos:

```
nodos = ["Depo", "ZA", "ZB", "ZC", "ZD", "C1", "C2", "C3"]
```

Aristas:

```
aristas = [("Depo", "ZA"), ("Depo", "ZB"), ("ZA", "ZC"),
            ("ZA", "ZD"), ("ZB", "ZD"), ("ZC", "C1"),
            ("ZD", "C2"), ("ZD", "C3")]
```

b) Lista y matriz de adyacencia.

```

from grafos import crear_listas_adyacencia_dirigido
from grafos2 import crear_matriz_adyacencia_dirigido

lista_ady = crear_listas_adyacencia_dirigido(nodos, aristas)
matriz_ady = crear_matriz_adyacencia_dirigido(nodos, aristas)

```

Lista de adyacencia (salida):

```

{'Depo': ['ZA', 'ZB'], 'ZA': ['ZC', 'ZD'], 'ZB': ['ZD'],
 'ZC': ['C1'], 'ZD': ['C2', 'C3'], 'C1': [], 'C2': [], 'C3': []}

```

Matriz de adyacencia (orden: Depo, ZA, ZB, ZC, ZD, C1, C2, C3):

	Depo	ZA	ZB	ZC	ZD	C1	C2	C3
Depo	0	1	1	0	0	0	0	0
ZA	0	0	0	1	1	0	0	0
ZB	0	0	0	0	1	0	0	0
ZC	0	0	0	0	0	1	0	0
ZD	0	0	0	0	0	0	1	1
C1	0	0	0	0	0	0	0	0
C2	0	0	0	0	0	0	0	0
C3	0	0	0	0	0	0	0	0

No es simétrica (las rutas son de ida), lo que confirma que es dirigido.

c) ¿Es un DAG? Justificar.

Sí. Es dirigido porque las aristas tienen dirección, van en un solo sentido (las rutas son de ida). Y es acíclico porque no se puede formar ningún ciclo: el flujo va del depósito a las zonas y de las zonas a los clientes, siempre hacia adelante, sin manera de retornar al punto de partida siguiendo las flechas. Al cumplir ambas condiciones, es un DAG.

d) Función que imprime los destinos directos de un nodo.

```

def destinos_directos(lista_ady, nodo):
    print(f"Desde {nodo} se puede llegar directamente a: {lista_ady[nodo]}")

for nodo in nodos:
    destinos_directos(lista_ady, nodo)

```

Salida:

```

Desde Depo se puede llegar directamente a: ['ZA', 'ZB']
Desde ZA se puede llegar directamente a: ['ZC', 'ZD']
Desde ZB se puede llegar directamente a: ['ZD']
Desde ZC se puede llegar directamente a: ['C1']
Desde ZD se puede llegar directamente a: ['C2', 'C3']
Desde C1 se puede llegar directamente a: []
Desde C2 se puede llegar directamente a: []
Desde C3 se puede llegar directamente a: []

```

e) Orden topológico válido.

Un orden posible en que las entregas respetan las dependencias:

Depo, ZA, ZB, ZC, ZD, C1, C2, C3

No es el único orden válido: como ZA y ZB solo dependen de Depo (y no entre sí), podrían intercambiarse; lo mismo ocurre entre los clientes finales.